



UNIVERSIDAD TÉCNICA
FEDERICO SANTA MARÍA

Documentación Tarea 3: Simulador Gráfico del Patrón Publicador-Suscriptor

Curso: Diseño y Programación Orientados a Objetos

Semestre: 1er Semestre 2025

Estudiantes:

- Sebastián Torres Valenta
- Ahmaru Oshien Huibidobro Garcia
- Antonio Alejandro Troncoso Bustos
- Martin Ignacio Cumbaynar Tabilo

Enlace del repositorio Github: https://github.com/sebadaba/2025-1_el0329_grupo-11_tarea-3

1. Diagrama UML de Clases

El siguiente diagrama muestra la arquitectura completa del simulador implementado en la Etapa 4.4:



2. Explicación de la Solución

2.1 Arquitectura General

La solución implementa el patrón de diseño **Publicador-Suscriptor** utilizando C++ y Qt para crear un simulador gráfico con dos tipos de contenido: videos y posiciones GPS.

2.2 Interacción entre Clases

Inicialización del Sistema

1. Broker se inicializa como el componente central que administra todos los tópicos del sistema.

2. Se crean dos Topic principales: "VideoTopic" y "GPSTopic".
3. Los Publisher (**VideoPublisher** y **GPSPublisher**) se registran con sus respectivos tópicos.
4. Los Subscriber (**VideoFollower** y **GPSFollower**) se suscriben a los tópicos correspondientes.

Flujo de Publicación de Videos

1. VideoPublisher recibe una URL a través de su **QLineEdit**.
2. Al presionar Enter, se invoca **onEnterPressed()** que llama a **publish(QString url)**.
3. El Publisher envía el mensaje al Broker especificando el tópico "VideoTopic".
4. Broker localiza el Topic correspondiente y ejecuta **notifySubscribers(QString message)**.
5. Topic itera sobre su lista de suscriptores y llama a **update(QString message)** en cada VideoFollower.
6. VideoFollower actualiza su interfaz gráfica, cambiando el texto del botón y preparando el **QMediaPlayer** para reproducir el video.

Flujo de Publicación GPS

1. GPSPublisher carga las posiciones desde un archivo usando **QFileDialog**.
2. QTimer se configura para ejecutar **onTimerTimeout()** cada 1000ms.
3. En cada timeout, se calcula la posición interpolada usando **interpolatePosition(double time)**.
4. La posición se publica siguiendo el mismo patrón: Publisher → Broker → Topic → Subscriber.
5. Cada GPSFollower recibe la actualización en su método **update(QString message)**.
6. GPSFollower parsea las coordenadas, actualiza sus variables internas y llama a **update()** para repintar la ventana.
7. **paintEvent(QPaintEvent*)** se ejecuta, dibujando un círculo en la nueva posición.

3. Dificultades Encontradas y Soluciones

3.1 Configuración de Módulos Qt para Multimedia

Dificultad: Al implementar la reproducción de video, el compilador generaba errores indicando que **QMediaPlayer** y **QVideoWidget** no estaban disponibles.

Causa: Los módulos de multimedia de Qt no estaban incluidos en el archivo de proyecto (.pro).

Solución: Se agregaron los módulos necesarios al archivo .pro:

```
QT += core widgets multimedia multimediawidgets
```

También se incluyeron los headers correspondientes en los archivos fuente:

```
#include <QMediaPlayer>
#include <QVideoWidget>
```

3.2 Conflictos de Nombres en Archivos UI

Dificultad: Error de compilación "invalid use of incomplete type class Ui::VideoForm" al intentar integrar las interfaces gráficas de video y GPS.

Causa: Ambas clases (`VideoFollower` y `GPSFollower`) estaban utilizando el mismo namespace `Ui::Form`, causando conflictos en la generación automática de código por Qt.

Solución: Se renombraron las clases UI en Qt Designer:

- Video: `objectName="VideoForm"` → genera `Ui::VideoForm`
- GPS: `objectName="GPSForm"` → genera `Ui::GPSForm`

Se actualizaron las declaraciones en los headers correspondientes y se ejecutó "Clean All" seguido de "Rebuild All" para regenerar los archivos `ui_*.h`.

3.3 Interpolación Linear de Posiciones GPS

Dificultad: Implementar la interpolación linear para generar posiciones intermedias cuando el archivo GPS contenía intervalos de tiempo mayores a 1 segundo.

Causa: Falta de comprensión inicial sobre cómo calcular posiciones intermedias usando interpolación linear basada en tiempo.

Solución: Se implementó el algoritmo de interpolación lineal en la función `pasarData`.

Se utilizó `QTimer` con intervalo de 1000ms para garantizar actualizaciones cada segundo, calculando las coordenadas X e Y independientemente para cada instante de tiempo.

4. Conclusiones

La implementación del patrón Publicador-Suscriptor en C++ con Qt demostró ser efectiva para crear un simulador gráfico robusto y extensible. La separación de responsabilidades entre las clases base abstractas y las implementaciones concretas permite fácil mantenimiento y extensión del sistema.

El uso de Qt facilitó significativamente el desarrollo de la interfaz gráfica y el manejo de multimedia, aunque requirió un período de aprendizaje para comprender correctamente la integración de sus diferentes módulos.

El desarrollo iterativo e incremental permitió identificar y resolver problemas de manera sistemática, resultando en una solución final que cumple con todos los requerimientos especificados en la tarea.